

Instruction Set Architecture (ISA)

RISC-V

→ *Estrutura de um programa Assembly*

RISC-V Instruction Set Architecture (ISA)

Diretivas de Assembler

- As diretivas não são instruções!
 - São comandos para o Assembler gerar o código binário

Diretiva	Descrição
<code>.text</code>	Declara uma zona de código (instruções)
<code>.data</code>	Declara uma zona de dados <i>writable</i>



Diretiva	Descrição
<code>.zero N</code>	Declara um vetor de N bytes inicializados a 0
<code>.byte num1[,num2, ...]</code>	Declara um vetor de bytes (1B) sequenciais em memória com valores num1, [num2, ...]
<code>.half num1[,num2, ...]</code>	Declara um vetor de half (2B) sequenciais em memória
<code>.word num1[,num2, ...]</code>	Declara um vetor de words (4B) sequenciais em memória
<code>.string "list of characters"</code>	Declara uma string em memória. Cada character (char) ocupa 1 B

Ripes Simulador

Estrutura de um
programa em
Assembly

Definem-se duas “zonas” no código Assembly: dados e código.

Declaração de constantes (equivalente a C defines)

```
.equ    MY_CONST,    213
```

Declaração de variáveis

```
.data
```

```
var1:   .word    153
```

```
var2:   .half    -1227, 3443, MY_CONST, 0x14, 13
```

```
str1:   .string  "cadeia de caracteres"
```

Código

```
.text
```

```
        la        x1, var1
```

```
        lw        x2, 0(x1)
```

```
loop:   j          loop
```

Ripes Simulador

Estrutura de um
programa em
Assembly

Definem-se duas “zonas” no código Assembly: dados e código.

Declaração de constantes (equivalente a C defines)

```
.equ MY_CONST, 213
```

Declaração de variáveis

```
.data
```

```
var1: .word 153
```

```
var2: .half -1227, 3443, MY_CONST, 0x14, 13
```

```
str1: .string "cadeia de caracteres"
```

Constantes

Nota: a vírgula após MY_CONST
é obrigatória

Código

```
.text
```

```
la x1, var1
```

```
lw x2, 0(x1)
```

```
loop: j loop
```

Ripes Simulador

Estrutura de um
programa em
Assembly

Definem-se duas “zonas” no código Assembly: dados e código.

Declaração de constantes (equivalente a C defines)

```
.equ MY_CONST, 213
```

Declaração de variáveis

```
.data
```

```
var1: .word 153
```

```
var2: .half -1227, 3443, MY_CONST, 0x14, 13
```

```
str1: .string "cadeia de caracteres"
```

Valores em
hexadecimal
(nomenclatura de C)

Código

```
.text
```

```
la x1, var1
```

```
lw x2, 0(x1)
```

```
loop: j loop
```

Ripes Simulador

Estrutura de um
programa em
Assembly

Definem-se duas “zonas” no código Assembly: dados e código.

Declaração de constantes (equivalente a C defines)

```
.equ    MY_CONST,    213
```

Declaração de variáveis

```
.data
```

```
var1:   .word    153
```

```
var2:   .half    -1227, 3443, MY_CONST, 0x14, 13
```

```
str1:   .string  "cadeia de caracteres"
```

Código

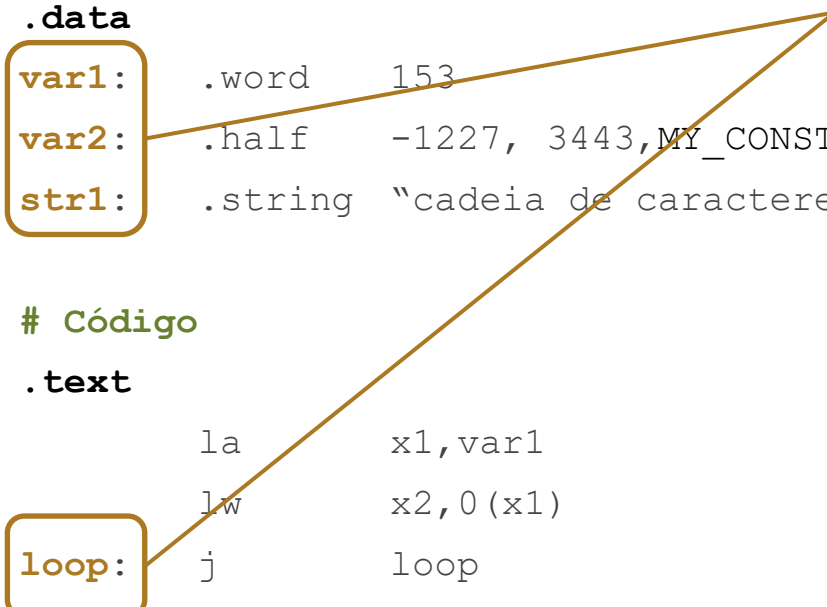
```
.text
```

```
la      x1, var1
```

```
lw      x2, 0(x1)
```

```
loop:   j        loop
```

Etiquetas(labels)



Ripes Simulador

Estrutura de um
programa em
Assembly

Definem-se duas “zonas” no código Assembly: dados e código.

Declaração de constantes (equivalente a C defines)

```
.equ MY_CONST, 213
```

Declaração de variáveis

```
.data
```

```
var1: .word 153
```

```
var2: .half -1227, 3443, MY_CONST, 0x14, 13
```

```
str1: .string "cadeia de caracteres"
```

Diretivas de Assembly

Código

```
.text
```

```
la x1, var1
```

```
lw x2, 0(x1)
```

```
loop: j loop
```

Instruções

Ripes Simulador

Estrutura de um
programa em
Assembly

Definem-se duas “zonas” no código Assembly: dados e código.

Declaração de constantes (equivalente a C defines)

```
.equ MY_CONST, 213
```

Declaração de variáveis

```
.data
```

```
var1: .word 153
```

```
var2: .half -1227, 3443, MY_CONST, 0x14, 13
```

```
str1: .string "cadeia de caracteres"
```

Código

```
.text
```

```
la x1, var1
```

```
lw x2, 0(x1)
```

```
loop: j loop
```

Operandos

Guia de Referência Rápido

Disponível na página da UC!!!

Não é suposto “decorar” os formatos e a codificação das instruções. É apenas preciso compreender.

O Quick Reference Guide pode ser consultado em todos os momentos de avaliação.

RV32IMFD Registers and Assembler Directives

Register	ABI Name	Description	Saver	Register	ABI Name	Description	Saver
x0	zero	Zero constant	—	f6-f7	f16-f17	FP temporaries	Caller
x1	ra	Return address	Caller	f8-f9	f50-f51	FP saved registers	Caller
x2	sp	Stack pointer	—	f10-f11	f50-f51	FP Funct. arguments/return values	Caller
x3	gp	Global pointer	—	f12-f17	f52-f57	FP Funct. arguments	Caller
x4	tp	Thread pointer	Caller	f18-f27	f52-f57	FP saved registers	Caller
x5-x7	t0-t2	Temporaries	Caller	f28-f31	f16-f17	FP temporaries	Caller
x8	s0 / fp	Saved / frame pointer	Caller	Assembler Directives:			
x9	s1	Saved register	Caller	• SIGMANTS:			
x10-x11	a0-a1	Funct. arguments/return values	Caller	• DATA (for RW data) and .text (for program)			
x12-x17	a2-a7	Funct. arguments	Caller	• PRE-INITIALIZE DATA:			
x18-x27	s2-s11	Saved registers	Caller	byte, half and word for a list of numbers with 1B, 2B and 4B			
x28-x31	t3-t6	Temporaries	Caller	.string to declare a string of characters			

Instruction Formats

R-Type	function7	rb	ra	function3	rd	opcode	0
S-Type	imm11[1:5]	rb	ra	function3	imm11[6:10]	opcode	
B-Type	imm11[12:10:5]	rb	ra	function3	imm11[11:11]	opcode	
U-Type		imm31[12]				opcode	
J-Type		imm20[10:11:11:19:12]				opcode	
R3-Type	function5	fmat	rb	ra	function3	rd	opcode
R4-Type	rc	fmat	rb	ra	function3	rd	opcode

RV32IM Instructions

	Instruction	Name	Type	Encoding	RTL Description
Move	nop	No Operation		Pseudo-inst.: addi x0, x0, 0	—
	li	Load (move) Immediate		Pseudo-inst.: li rd, imm(symbol)	rd ← imm (symbol)
	la	Load Address		U 010111	rd ← imm << 12
	lui	Load Upper Imm		U 010111	rd ← PC + (imm << 12)
	auipc	Add Upper Imm to PC		U 001011	rd ← xs
Arithmetic, Logic, Shift	add	Add Register Value		Pseudo-inst.: addi rd, xs, x0	rd ← xs
	neg	Negate (2's complement)		Pseudo-inst.: sub rd, xs, x0	rd ← -xs
	addi	ADD Immediate	R	0100011 0x0	rd ← xs + x0
	sub	SUB Immediate	R	0110011 0x0	rd ← xs - x0
	not	NOT		Pseudo-inst.: xori rd, xs, -1	rd ← ~xs
Multiply, Divide	and	AND	R	0100011 0x7	rd ← xs & x0
	andi	AND Immediate	I	0100011 0x7	rd ← xs & imm
	or	OR	R	0100011 0x6	rd ← xs x0
	ori	OR Immediate	I	0100011 0x6	rd ← xs imm
	xor	XOR	R	0100011 0x4	rd ← xs ^ x0
Fixed	xori	XOR Immediate	I	0100011 0x4	rd ← xs ^ imm
	sll	Shift Left Logical	R	0100011 0x1	rd ← xs << x0[5:0]
	slli	Shift Left Logical Imm	I	0100011 0x1	rd ← xs << x0[5:0]
	srl	Shift Right Logical	R	0100011 0x5	rd ← xs >> x0[5:0]
	srlw	Shift Right Logical Imm	I	0100011 0x5	rd ← xs >> x0[5:0]
Convert	sra	Shift Right Arithmetic	R	0100011 0x5	rd ← xs >> x0[5:0]
	srai	Shift Right Arith Imm	I	0100011 0x5	rd ← xs >> x0[5:0]
	mul	Multiply	R	0100011 0x0	rd ← (xs × x0)[31:0]
	mulh	Multiply High (S×S)	R	0100011 0x1	rd ← (xs × x0)[32:16]
	mulhsu	Multiply High (S×U)	R	0100011 0x2	rd ← (xs × x0)[32:16]
Load, Store	mulu	Multiply High (U×U)	R	0100011 0x3	rd ← (xs × x0)[32:16]
	div	Divide	R	0100011 0x4	rd ← xs / x0
	divu	Divide (U)	R	0100011 0x5	rd ← xs / x0
	rem	Remainder	R	0100011 0x6	rd ← xs % x0
	remu	Remainder (U)	R	0100011 0x7	rd ← xs % x0
Compare	lb	Load Byte	R	0000011 0x1	rd ← M[xs+imm] (B)
	lh	Load Halfword	R	0000011 0x1	rd ← M[xs+imm] (H)
	lw	Load Word	R	0000011 0x2	rd ← M[xs+imm] (W)
	lbu	Load Byte (U)	R	0000011 0x2	rd ← M[xs+imm] (BU)
	lhu	Load Halfword (U)	R	0000011 0x2	rd ← M[xs+imm] (HU)
Store	sb	Store to Global Symbol		Pseudo-inst.: auipc + 32 0[h]w	M[xs+imm] ← x0[h]w
	sh	Store Byte	S	0100011 0x0	M[xs+imm] ← x0[7:0]
	shw	Store Half	S	0100011 0x1	M[xs+imm] ← x0[15:0]
	sw	Store Word	S	0100011 0x2	M[xs+imm] ← x0[31:0]

	Instruction	Name	Type	Encoding	RTL Description
Compare	slt	Set Less Than	R	0100011 0x2	rd ← (xs < x0)?1:0
	slti	Set Less Than Imm	I	0010011 0x2	rd ← (xs < imm)?1:0
	sltu	Set Less Than (U)	R	0110011 0x3	rd ← (xs < imm)?1:0
	sltiu	Set Less Than Imm (U)	I	0010011 0x3	rd ← (xs < imm)?1:0
	segt	Set Equal Zero		Pseudo-inst.: sltu rd, xs, 1	rd ← (xs == 0)?1:0
Flow control (branch, jump, call, ret)	snez	Set Not Equal Zero		Pseudo-inst.: sltu rd, x0, xs	rd ← (xs != 0)?1:0
	slgt	Set Less Than Zero		Pseudo-inst.: slt rd, xs, x0	rd ← (xs < 0)?1:0
	sgt	Set Greater Than Zero		Pseudo-inst.: slt rd, x0, xs	rd ← (xs < 0)?1:0
	breq	Branch Equal	B	1100011 0x0	if(xa==xb) PC ← PC + imm
	bne	Branch Not Equal	B	1100011 0x1	if(xa!=xb) PC ← PC + imm
OS	bgt	Branch Greater Than		Pseudo-inst.: blt rd, xs, offset	if (xs>xt) PC ← symbol
	bge	Branch Greater or Equal		Pseudo-inst.: blt rd, xs, offset	if (xs>=xt) PC ← symbol
	ble	Branch Less or Equal		Pseudo-inst.: bge rd, xs, offset	if (xs<=xt) PC ← symbol
	blt	Branch Less Than	B	1100011 0x4	if(xa<xb) PC ← PC + imm
	bgtu	Branch Less Than (U)		Pseudo-inst.: bltu rd, xs, offset	if (xs>xt) PC ← symbol
OS	bgeu	Branch Less or Equal (U)		Pseudo-inst.: bltu rd, xs, offset	if (xs>=xt) PC ← symbol
	bleu	Branch Less or Equal (U)		Pseudo-inst.: bgtu rd, xs, offset	if (xs<=xt) PC ← symbol
	bneq	Branch Not Equal Zero		Pseudo-inst.: beq xs, x0, offset	if (xs!=0) PC ← symbol
	bnez	Branch Not Equal Zero		Pseudo-inst.: bne xs, x0, offset	if (xs!=0) PC ← symbol
	blez	Branch Less or Equal Zero		Pseudo-inst.: bge xs, x0, offset	if (xs<=0) PC ← symbol
OS	bgez	Branch Greater or Equal Zero		Pseudo-inst.: blt xs, x0, offset	if (xs>0) PC ← symbol
	bltz	Branch Less Than Zero		Pseudo-inst.: blt xs, x0, offset	if (xs<0) PC ← symbol
	j	Jump	J	1101111 0x0	PC ← symbol
	jal	Jump And Link	J	1101111 0x1	rd ← PC+4; PC ← PC + imm
	jalu	Jump And Link To Symbol		Pseudo-inst.: jal rd, xs, 0	rd ← PC + 4; PC ← symbol
OS	jalr	Jump Register		Pseudo-inst.: jalr xt, xs, 0	rd ← PC + 4; PC ← xs
	jalrw	Jump And Link Register		Pseudo-inst.: jalr xt, xs, 0	rd ← PC + 4; PC ← xs
	call	Call subroutine	I	1101111 0x0	rd ← PC+4; PC ← xs + imm
	ret	Return from subroutine		Pseudo-inst.: jalr x0, x1, 0	PC ← x1
	ecall	Environment Call	I	1110011 0x0	imm=0x0
	ebreak	Environment Break	I	1110011 0x0	imm=0x1

RV32FD Floating Point Instructions (not available on Ripes)

	(F/D) Instruction	Name	Type	Encoding	RTL Description
LD, ST	flw	FP Load from Symbol	I	Pseudo-inst.: auipc + flw[1:5]	fd ← M[symbol]
	flwu	FP Load from Symbol	I	Pseudo-inst.: auipc + flw[1:5]	fd ← M[symbol]
	fsd	FP Store to Symbol	S	Pseudo-inst.: auipc + fsd[1:5]	M[symbol] ← fb (modifies xt)
	fsdu	FP Store to Symbol	S	Pseudo-inst.: auipc + fsd[1:5]	M[symbol] ← fb
	fsw	FP Store	S	Pseudo-inst.: auipc + fsd[1:5]	M[symbol] ← fb
Move	fmv	FP Sign Injection		Pseudo-inst.: fsgnj, (sd) rd, fs, fs	fd = fs
	fsgnj	FP Sign Injection	R3	1010011 000	fd = abs(fs) + sgn(fb)
	fsgnjw	FP Sign Neg Injection	R3	1010011 001	fd = abs(fs) + -sgn(fb)
	fsgnjx	FP Sign Xor Injection	R3	1010011 010	fd = fs + sgn(fb)
	fabs	FP Absolute Value		Pseudo-inst.: fsgnj, (sd) rd, fs, fs	fd = fs
Arithmetic	fneg	FP Negative Value		Pseudo-inst.: fsgnj, (sd) rd, fs, fs	fd = -fs
	fmin	FP Minimum	R3	1010011 000	fd = min(fs, fb)
	fmax	FP Maximum	R3	1010011 001	fd = max(fs, fb)
	fadd	FP Add	R3	1010011 000	fd = fs + fb
	fsub	FP Sub	R3	1010011 001	fd = fs - fb
Fixed	fmul	FP Mul	R3	1010011 000	fd = fs * fb
	fdiv	FP Div	R3	1010011 001	fd = fs / fb
	fsqrt	FP Square Root	R3	1010011 000	fd = sqrt(fs)
	fmsadd	FP Fused Mnl-Add	R4	1000011 000	fd = fs + fb + fc
	fmsub	FP Fused Mnl-Sub	R4	1000011 001	fd = fs + fb - fc
Convert	fmsadd	FP Neg Fused Mnl-Add	R4	1000011 000	fd = -fs + fb + fc
	fmsub	FP Neg Fused Mnl-Sub	R4	1000011 001	fd = -fs + fb - fc
	fcvt	FP Conv from Sign Int	R3	1010011 000	fd = (f16to32) fs
	fcvt	FP Conv from Uns Int	R3	1010011 001	fd = (f16to32) fs
	fcvt	FP Conv to Int	R3	1010011 000	xd = (int32_t) fs
Compare	fcvt	FP Conv to Int	R3	1010011 001	xd = (uint32_t) fs
	fcvt	FP Conv to Int	R3	1010011 000	xd = (int32_t) fs
	fcvt	FP Conv to Int	R3	1010011 001	xd = (uint32_t) fs
	fcvt	FP Conv to Int	R3	1010011 000	xd = (int32_t) fs
	fcvt	FP Conv to Int	R3	1010011 001	xd = (uint32_t) fs

Observation: Designations flw and fsd represent the corresponding instructions for SP FP load/store (flw/fsd) and for DP FP load/store (flw/fsd). Similarly, designations fsw and fsdu represent the corresponding SP FP store and DP FP store instructions.

Guia de Referência Rápido

Disponível na página da UC!!!

Não é suposto “decorar” os formatos e a codificação das instruções. É apenas preciso compreender.

**O Quick Reference Guide
pode ser consultado em
todos os momentos de
avaliação.**

Assembler Directives:

- DECLARATION OF SEGMENTS:
 .data (for RW data) and .text (for program)
 - DECLARATION OF DATA (WITHIN .data SEGMENT):
 .byte, .half, .word, .string – initialized list of values/characters
 .zero – N bytes initialized with value 0
-

Guia de Referência Rápido

Disponível na página da UC!!!

Não é suposto “decorar” os formatos e a codificação das instruções. É apenas preciso compreender.

O Quick Reference Guide pode ser consultado em todos os momentos de avaliação.

RV32IM Instructions

	Instruction	Name	Encoding				RTL Description
			Type	Opcode	funct3	funct7	
Move	nop	No Operation	<i>Pseudo-inst.</i> : addi x0,x0,0				-
	li xd,imm	Load (move) Immediate	<i>Pseudo-inst.</i> : Myriad sequence				$xd \leftarrow imm$
	la xd,symbol	Load Address	<i>Pseudo-inst.</i> : auipc + addi				$xd \leftarrow imm \text{ (symbol)}$
	lui xd,imm	Load Upper Imm	U	0110111			$xd \leftarrow imm << 12$
	auipc xd,imm	Add Upper Imm to PC	U	0010111			$xd \leftarrow PC + (imm << 12)$
Arithmetic, Logic, Shift	mv xd,rs	Move Register Value	<i>Pseudo-inst.</i> : addi xd, xs, 0				$xd \leftarrow xs$
	neg xd,xs	Negate (2's complement)	<i>Pseudo-inst.</i> : sub xd, x0, xs				$xd \leftarrow -xs$
	add xd,xa,xs	ADD	R	0110011	0x0	0x00	$xd \leftarrow xa + xs$
	addi xd,xa,imm	ADD Immediate	I	0010011	0x0		$xd \leftarrow xa + imm$
	sub xd,xa,xs	SUB	R	0110011	0x0	0x20	$xd \leftarrow xa - xs$
	not xd,xs	NOT	<i>Pseudo-inst.</i> : xori xd, xs, -1				$xd \leftarrow \sim xs$
	and xd,xa,xs	AND	R	0110011	0x7	0x00	$xd \leftarrow xa \& xs$
	andi xd,xa,imm	AND Immediate	I	0010011	0x7		$xd \leftarrow xa \& imm$
	or xd,xa,xs	OR	R	0110011	0x6	0x00	$xd \leftarrow xa xs$
	ori xd,xa,imm	OR Immediate	I	0010011	0x6		$xd \leftarrow xa imm$
	xor xd,xa,xs	XOR	R	0110011	0x4	0x00	$xd \leftarrow xa \wedge xs$
	xori xd,xa,imm	XOR Immediate	I	0010011	0x4		$xd \leftarrow xa \wedge imm$
	sll xd,xa,xs	Shift Left Logical	R	0110011	0x1	0x00	$xd \leftarrow xa << xs_{[4:0]}$
	slli xd,xa,imm	Shift Left Logical Imm	I	0010011	0x1		$xd \leftarrow xa << imm_{[4:0]}$
	srl xd,xa,xs	Shift Right Logical	R	0110011	0x5	0x00	$xd \leftarrow xa >> xs_{[4:0]}$
	srlr xd,xa,imm	Shift Right Logical Imm	I	0010011	0x5		$xd \leftarrow xa >> imm_{[4:0]}$
	sra xd,xa,xs	Shift Right Arithmetic	R	0110011	0x5	0x20	$xd \leftarrow xa >> xs_{[4:0]}$
	srai xd,xa,imm	Shift Right Arith Imm	I	0010011	0x5		$xd \leftarrow xa >> imm_{[4:0]}$

Guia de Referência Rápido

Disponível na página da UC!!!

Não é suposto “decorar” os formatos e a codificação das instruções. É apenas preciso compreender.

O Quick Reference Guide pode ser consultado em todos os momentos de avaliação.

RV32IM Instructions

	Instruction	Name	Encoding				RTL Description
			Type	Opcode	funct3	funct7	
Move	nop	No Operation	<i>Pseudo-inst.</i> : addi x0,x0,0				-
	li xd,imm	Load (move) Immediate	<i>Pseudo-inst.</i> : Myriad sequence				$xd \leftarrow imm$
	la xd,symbol	Load Address	<i>Pseudo-inst.</i> : auipc + addi				$xd \leftarrow imm (symbol)$
	lui xd,imm	Load Upper Imm	U	0110111			$xd \leftarrow imm << 12$
	auipc xd,imm	Add Upper Imm to PC	U	0010111			$xd \leftarrow PC + (imm << 12)$
Arithmetic, Logic, Shift	mv xd,rs	Move Register Value	<i>Pseudo-inst.</i> : addi xd, xs, 0				$xd \leftarrow xs$
	neg xd,xs	Negate (2's complement)	<i>Pseudo-inst.</i> : sub xd, x0, xs				$xd \leftarrow -xs$
	add xd,xa,xb	ADD	R	0110011	0x0	0x00	$xd \leftarrow xa + xb$
	addi xd,xa,imm	ADD Immediate	I	0010011	0x0		$xd \leftarrow xa + imm$
	sub xd,xa,xb	SUB	R	0110011	0x0	0x20	$xd \leftarrow xa - xb$
	not xd,xs	NOT	<i>Pseudo-inst.</i> : xori xd, xs, -1				$xd \leftarrow \sim xs$
	and xd,xa,xb	AND	R	0110011	0x7	0x00	$xd \leftarrow xa \& xb$
	andi xd,xa,imm	AND Immediate	I	0010011	0x7		$xd \leftarrow xa \& imm$
	or xd,xa,xb	OR	R	0110011	0x6	0x00	$xd \leftarrow xa xb$
	ori xd,xa,imm	OR Immediate	I	0010011	0x6		$xd \leftarrow xa imm$
	xor xd,xa,xb	XOR	R	0110011	0x4	0x00	$xd \leftarrow xa \wedge xb$
	xori xd,xa,imm	XOR Immediate	I	0010011	0x4		$xd \leftarrow xa \wedge imm$
	sll xd,xa,xb	Shift Left Logical	R	0110011	0x1	0x00	$xd \leftarrow xa << xb_{[4:0]}$
	slli xd,xa,imm	Shift Left Logical Imm	I	0010011	0x1		$xd \leftarrow xa << imm_{[4:0]}$
	srl xd,xa,xb	Shift Right Logical	R	0110011	0x5	0x00	$xd \leftarrow xa >> xb_{[4:0]}$
	srli xd,xa,imm	Shift Right Logical Imm	I	0010011	0x5		$xd \leftarrow xa >> imm_{[4:0]}$
	sra xd,xa,xb	Shift Right Arithmetic	R	0110011	0x5	0x20	$xd \leftarrow xa >> xb_{[4:0]}$
	srai xd,xa,imm	Shift Right Arith Imm	I	0010011	0x5		$xd \leftarrow xa >> imm_{[4:0]}$

Emulador RISC-V (Ripes)

intro.s aberto no Ripes

Ficheiro intro.s
disponibilizado para o
laboratório 0

Source code Input type: ☒ Assembly ☐ C Executable code View mode: ☐ Binary ☒ Disassembled

```
1 # Declaracao de variaveis
2 .data
3 var1:    .word    153, 0b11, 0xf
4 var2:    .half    -1227, 3443, 213, 0x14, 13
5 var3:    .byte    255, 128, 0, 1
6 table1:  .zero    16
7 str1:    .string  "Introducao ao RIPES"
8
9 # Codigo
10 .text
11      la      x5, var1
12      lw      x6, 0(x5)
13      addi    x7, x6, 1
14      sw      x7, 0(x5)
15
16      la      x10, str1
17      li      x17, 4
18      ecall
19
20 loop:  j      loop
21
```

```
0:      10000297      auipc x5 0x10000
4:      00028293      addi x5 x5 0
8:      0002a303      lw x6 0 x5
c:      00130393      addi x7 x6 1
10:     0072a023      sw x7 0 x5
14:     10000517      auipc x10 0x10000
18:     01650513      addi x10 x10 22
1c:     00400893      addi x17 x0 4
20:     00000073      ecall

00000024 <loop>:
24:     0000006f      jal x0 0 <loop>
```

Guia de Referência Rápido

Disponível na página da UC!!!

Não é suposto “decorar” os formatos e a codificação das instruções. É apenas preciso compreender.

O Quick Reference Guide pode ser consultado em todos os momentos de avaliação.

RV32IM Instructions

	Instruction	Name	Encoding				RTL Description
			Type	Opcode	funct3	funct7	
Move	nop	No Operation	Pseudo-inst.: addi x0,x0,0				-
	li xd,imm	Load (move) Immediate	Pseudo-inst.: Myriad sequence				$xd \leftarrow imm$
	la xd,symbol	Load Address	Pseudo-inst.: auipc + addi				$xd \leftarrow imm (symbol)$
	lui xd,imm	Load Upper Imm	U	0110111			$xd \leftarrow imm << 12$
	auipc xd,imm	Add Upper Imm to PC	U	0010111			$xd \leftarrow PC + (imm << 12)$
Arithmetic, Logic, Shift	mv xd,rs	Move Register Value	Pseudo-inst.: addi xd, xs, 0				$xd \leftarrow xs$
	neg xd,xs	Negate (2's complement)	Pseudo-inst.: sub xd, x0, xs				$xd \leftarrow -xs$
	add xd,xa,xb	ADD	R	0110011	0x0	0x00	$xd \leftarrow xa + xb$
	addi xd,xa,imm	ADD Immediate	I	0010011	0x0		$xd \leftarrow xa + imm$
	sub xd,xa,xb	SUB	R	0110011	0x0	0x20	$xd \leftarrow xa - xb$
	not xd,xs	NOT	Pseudo-inst.: xori xd, xs, -1				$xd \leftarrow \sim xs$
	and xd,xa,xb	AND	R	0110011	0x7	0x00	$xd \leftarrow xa \& xb$
	andi xd,xa,imm	AND Immediate	I	0010011	0x7		$xd \leftarrow xa \& imm$
	or xd,xa,xb	OR	R	0110011	0x6	0x00	$xd \leftarrow xa xb$
	ori xd,xa,imm	OR Immediate	I	0010011	0x6		$xd \leftarrow xa imm$
	xor xd,xa,xb	XOR	R	0110011	0x4	0x00	$xd \leftarrow xa \wedge xb$
	xori xd,xa,imm	XOR Immediate	I	0010011	0x4		$xd \leftarrow xa \wedge imm$
	sll xd,xa,xb	Shift Left Logical	R	0110011	0x1	0x00	$xd \leftarrow xa << xb_{[4:0]}$
	slli xd,xa,imm	Shift Left Logical Imm	I	0010011	0x1		$xd \leftarrow xa << imm_{[4:0]}$
	srl xd,xa,xb	Shift Right Logical	R	0110011	0x5	0x00	$xd \leftarrow xa >> xb_{[4:0]}$
	srlr xd,xa,imm	Shift Right Logical Imm	I	0010011	0x5		$xd \leftarrow xa >> imm_{[4:0]}$
	sra xd,xa,xb	Shift Right Arithmetic	R	0110011	0x5	0x20	$xd \leftarrow xa >> xb_{[4:0]}$
	srai xd,xa,imm	Shift Right Arith Imm	I	0010011	0x5		$xd \leftarrow xa >> imm_{[4:0]}$

Emulador RISC-V (Ripes)

intro.s aberto no Ripes

Ficheiro intro.s
disponibilizado para o
laboratório 0

Source code Input type: ☒ Assembly ☐ C Executable code View mode: ☐ Binary ☒ Disassembled

```
1 # Declaracao de variaveis
2 .data
3 var1:    .word    153, 0b11, 0xf
4 var2:    .half    -1227, 3443, 213, 0x14, 13
5 var3:    .byte    255, 128, 0, 1
6 table1:  .zero    16
7 str1:    .string  "Introducao ao RIPES"
8
9 # Codigo
10 .text
11      la      x5, var1
12      lw      x6, 0(x5)
13      addi    x7, x6, 1
14      sw      x7, 0(x5)
15
16      la      x10, str1
17      li      x17, 4
18      ecall
19
20 loop:  j      loop
21
```

0:	10000297	auipc x5 0x10000
4:	00028293	addi x5 x5 0
8:	0002a303	lw x6 0 x5
c:	00130393	addi x7 x6 1
10:	0072a023	sw x7 0 x5
14:	10000517	auipc x10 0x10000
18:	01650513	addi x10 x10 22
1c:	00400893	addi x17 x0 4
20:	00000073	ecall

00000024 <loop>:		
24:	0000006f	jal x0 0 <loop>

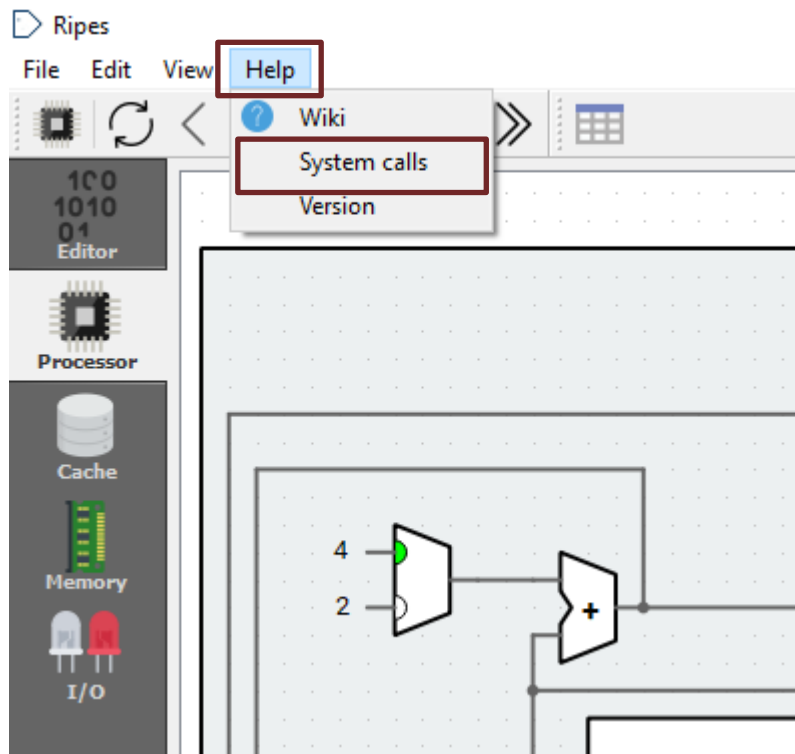
Emulador RISC-V (Ripes)

Chamadas de sistema

a7	a0	Name	Description
1	(integer to print)	print_int	Prints the value located in a0 as a signed integer
2	(float to print)	print_float	Prints the value located in a0 as a floating point number
4	(pointer to string)	print_string	Prints the null-terminated string located at address in a0
10	-	exit	Halts the simulator
11	(char to print)	print_char	Prints the value located in a0 as an ASCII character
34	(integer to print)	print_hex	Prints the value located in a0 as a hex number
35	(integer to print)	print_bin	Prints the value located in a0 as a binary number
36	(integer to print)	print_unsigned	Prints the value located in a0 as an unsigned integer
93	(status code)	exit	Halts the simulator and exits with status code in a0

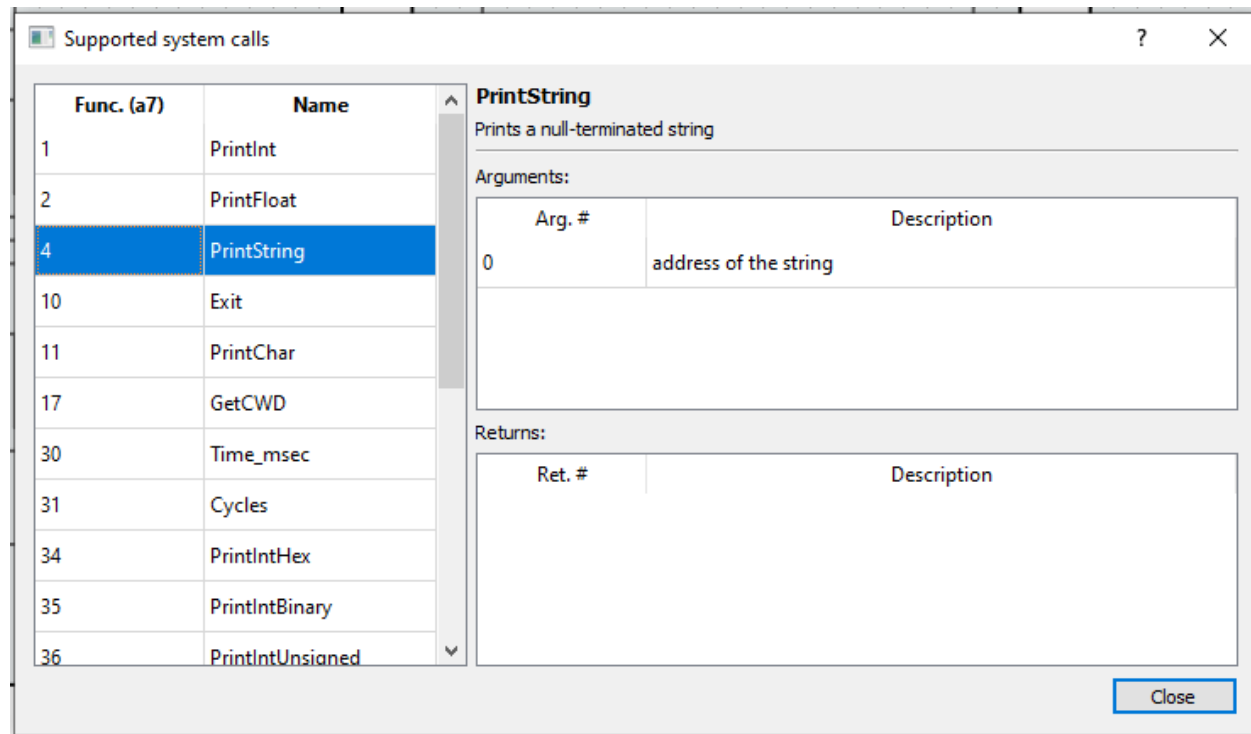
Emulador RISC-V (Ripes)

Chamadas de sistema



Emulador RISC-V (Ripes)

Chamadas de sistema



Guia de Referência Rápido

Registos e os seus alias

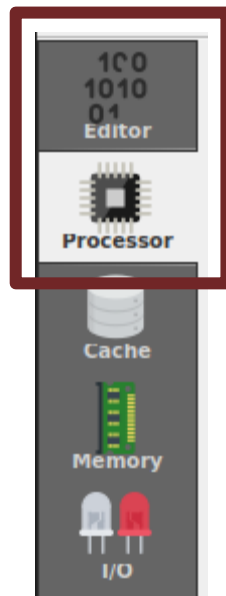
RV32IMFD Registers

Register	ABI Name	Description	Saver*
x0	zero	Zero constant	—
x1	ra	Return address	Caller
x2	sp	Stack pointer	—
x3	gp	Global pointer	—
x4	tp	Thread pointer	Callee
x5-x7	t0-t2	Temporaries	Caller
x8	s0 / fp	Saved / frame pointer	Callee
x9	s1	Saved register	Callee
x10-x11	a0-a1	Funct. arguments/return values	Caller
x12-x17	a2-a7	Funct. arguments	Caller
x18-x27	s2-s11	Saved registers	Callee
x28-x31	t3-t6	Temporaries	Caller

* Within a given function f, all registers marked as caller are considered temporary and do not need to be saved. All registers marked as callee must be saved at function entrance, if used.

Emulador RISC-V (Ripes)

Registos e os seus alias



GPR		
Name	Alias	Value
x0	zero	0x00000000
x1	ra	0x00000000
x2	sp	0x7fffffff0
x3	gp	0x10000000
x4	tp	0x00000000
x5	t0	0x00000000
x6	t1	0x00000000
x7	t2	0x00000000
x8	s0	0x00000000
x9	s1	0x00000000
x10	a0	0x00000000
x11	a1	0x00000000
x12	a2	0x00000000
x13	a3	0x00000000
x14	a4	0x00000000
x15	a5	0x00000000
x16	a6	0x00000000
x17	a7	0x00000000
x18	s2	0x00000000

Display type: Hex